

```
if k%2 == 0:
    colors2.append('red')
else:
    colors2.append('blue')
lc = mc.LineCollection(game, colors = colors2, linewidths = 2)
fig, ax = plt.subplots()
for i in range(1,n + 1):
    for j in range(1,n + 1):
        plt.plot(i,j,'o',c = 'black')
ax.add_collection(lc)
ax.autoscale()
ax.margins(0.1)
plt.savefig(name)
```

Listing 10-1: *A function that draws a game board for dots and boxes*

This function takes `n` and `name` as arguments, just as `drawlattice()` did. It also includes exactly the same nested loops we used to draw lattice points in `drawlattice()`. The first addition you can see is the `colors2` list, which starts out empty, and we fill it up with the colors we assign to the line segments that we'll draw. In dots and boxes, turns alternate between the two players, so we'll alternate the colors of the line segments that we assign to the players—in this case, red for the first player and blue for the second player. The `for` loop after the definition of the `colors2` list fills it up with alternating instances of `'red'` and `'blue'` until there are as many color assignments as there are moves in the game. The other lines of code we've added create a collection of lines out of our game moves and draw them, in the same way we've drawn collections of lines in previous chapters.

NOTE

This book is not printed in color, and it's not totally necessary to have any colors when you play dots and boxes. But the code for colors is included anyway so you can see them when you run the code at home.

We can call our `drawgame()` function in one line as follows:

```
drawgame(5, 'gameinprogress.png', game)
```

This is exactly how we created [Figure 10-2](#).